

7/23/23, 3:22 AM

Codio - Coding Project

CodioProjectFileEditFindViewToolsEducationHelpConfigure...Project Index (static)Configure...

SANDIPAN DEY

app.py

```
17 # Create a chat completion request using OpenAI's GPT-3.5 Turbo model
18 response = openai.ChatCompletion.create(
19     model="gpt-3.5-turbo",
20     messages=[
21         {"role": "system", "content": "You are an AI assistant."},
22         {"role": "user", "content": prompt}
23     ]
24 )
25 # Extract the generated chat response from the API response
26 chat_response = response['choices'][0]['message']['content'].strip()
27 return chat_response
28
29 # Route for handling chat requests
30 @app.route("/chat", methods=["POST"])
31 def chat():
32     user_input = request.json["user_input"] # Get the user input from the request
33     # Generate chatbot response using the user input
34     chat_response = generate_chat_response(user_input)
35     response_data = {"response": chat_response}
36     return jsonify(response_data)
37
38 # Routes for the home page
39 @app.route("/")
40 @app.route("/index.html")
41 def index():
42     return render_template("index.html") # Render the index.html template
43
44 if __name__ == "__main__":
45     app.run(host='0.0.0.0') # Run the Flask application on the host '0.0.0.0'
```

100% (1:1)

Terminal

```
codio@accentlegacy-legalfactor:~/workspace$
```

Guide

Collapse

Building the Server-Side Code

Next, we'll use Flask to create a simple server that receives user input and sends requests to the ChatGPT API.

This is not a flask course, we will provide the flask connection for this part of the assignment. However, since this is Python code it is recommended to read (also it is commented). Here's the code for our Flask server:

```
import openai
from flask import Flask, jsonify, request, render_template
import os

# Set environment variables
keys = os.getenv('OPENAI_KEY') # Get the OpenAI API key from environment variable

api_key = keys

app = Flask(__name__)
openai.api_key = api_key

# Function to generate chatbot response
def generate_chat_response(user_input):
    prompt = user_input
    # Create a chat completion request using OpenAI's GPT-3.5 Turbo model
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are an AI assistant."},
            {"role": "user", "content": prompt}
        ]
    )
    # Extract the generated chat response from the API response
    chat_response = response['choices'][0]['message']['content'].strip()
    return chat_response

# Route for handling chat requests
@app.route("/chat", methods=["POST"])
def chat():
    user_input = request.json["user_input"] # Get the user input from the request
    # Generate chatbot response using the user input
    chat_response = generate_chat_response(user_input)
    response_data = {"response": chat_response}
    return jsonify(response_data)

# Routes for the home page
@app.route("/")
@app.route("/index.html")
def index():
    return render_template("index.html") # Render the index.html template

if __name__ == "__main__":
    app.run(host='0.0.0.0') # Run the Flask application on the host '0.0.0.0'
```

In this Flask server code, we define a function `generate_chat_response` that takes a user input and sends a request to the ChatGPT API to generate a response. We then define a Flask route `/chat` that handles POST requests with the user input in the request body. The route function calls the `generate_chat_response` function and returns the response in JSON format.

To run the Flask server, we can add the following lines of code to the end of our Python file:

```
if __name__ == "__main__":
    app.run(host='0.0.0.0')
```

This will start the Flask server in debug mode and allow us to test our application.

Now that we have both our user interface and server-side code in place, we can run our web application and test it out. To start the application, we can run the following command in our terminal:

```
python3 app.py
```

This will start the Flask server. You can click the link below to open it inside Codio(top left panel).

Open website

When we enter text in the user input field and submit the form, our server-side code will send a request to the ChatGPT API and display the response in the response text area. Overall, Your program should run something like the gif below.

ChatGPT API Demo

User Input:

Enter text

Submit

Response:

```
Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

Building the Server-Side Code

python3 app.py

This will start the Flask server, and we can navigate to <http://0.0.0.0:5000/> web browser to view the website. Based on our browser settings, you can click the link below to open it inside Codio.

When we enter text in the user input field and submit the form, our code will send a request to the ChatGPT API and display the response in the response text area.

What HTTP method does the "/chat" route handle in the application?

☐ GET

☒ POST

☐ PUT

☐ DELETE

What HTTP method does the `/chat` route handle in the Flask application?

☐ GET

☒ POST

☐ PUT

☐ DELETE

The `/chat` route is defined with the decorator `@app.route("/chat", methods=["POST"])`, which means it is set up to handle POST requests.

Check !!!

Next

1/1